

Le modèle des Transformers pour l'analyse d'image

Anrezki Ayoub

Résumé

Les Transformers, introduits initialement pour le traitement du langage, sont depuis devenus une architecture centrale en analyse d'image, à travers notamment le Vision Transformer. Ce papier propose une présentation rigoureuse de cette architecture, destinée à un lecteur formé aux mathématiques mais étranger au domaine, où chaque objet est introduit par le besoin précis auquel il répond plutôt que posé sans justification. Nous partons du cadre statistique de l'apprentissage automatique pour motiver l'apparition du mécanisme d'attention, avant de construire l'architecture Transformer dans son intégralité (attention, encodage positionnel, architecture complète), puis son adaptation à l'analyse d'image. Tout au long du texte, nous distinguons explicitement trois régimes : les faits calculatoires, vérifiables directement ; les théorèmes établis dans la littérature, que nous prouvons ou admettons honnêtement avec référence selon leur portée ; et les choix d'architecture motivés par l'observation empirique plutôt que par une démonstration, présentés comme tels. Une courte excursion vers les grands modèles de langage complète le tableau.

Table des matières

1	Introduction	1
2	Préliminaires	2
2.1	Le problème de la règle inconnue	2
2.2	Le cadre statistique de l'apprentissage	2
2.2.1	Représenter les entrées et les sorties	2
2.2.2	Pourquoi une loi de probabilité plutôt qu'une fonction	3
2.2.3	Espace des hypothèses, perte, risque	3
2.3	Optimisation	4
2.3.1	Le problème que l'optimisation doit résoudre	4
2.3.2	La descente de gradient	5
2.3.3	Pourquoi la descente stochastique (SGD)	5
2.3.4	Calculer $\nabla_{\theta}\mathcal{L}$: la rétropropagation	6

1 Introduction

Les Transformers, introduits par Vaswani et al. (2017), sont aujourd'hui l'architecture dominante en traitement du langage et, depuis les Vision Transformers, en analyse d'image. Pourtant, la plupart des présentations disponibles admettent l'architecture plus qu'elles ne la démontrent : les objets mathématiques sous-jacents (produit scalaire, softmax, espaces de représentation) sont rarement posés avec la rigueur qu'un lecteur habitué à la preuve est en droit d'attendre.

Ce papier tente de combler cet écart, en distinguant à chaque étape ce qui est un fait démontrable, ce qui est un théorème établi ailleurs et admis avec référence, et ce qui n'est qu'un choix d'architecture motivé empiriquement.

2 Préliminaires

2.1 Le problème de la règle inconnue

Il existe des tâches pour lesquelles on connaît une suite finie d'instructions qui, à partir d'une entrée, produit la sortie voulue, et dont on peut prouver la correction : trier une liste, tester la primalité d'un entier, ou encore calculer la dérivée d'une expression symbolique donnée sous forme d'un arbre syntaxique explicite (une composition d'opérations élémentaires). Dans ce dernier cas, la dérivation formelle procède par récurrence structurelle sur l'arbre (linéarité pour une somme, règle du produit, règle de la chaîne pour une composition, table pour les fonctions atomiques), et sa correction se prouve par induction structurelle. Aucun apprentissage n'est nécessaire : la règle est connue et prouvée.

Il existe d'autres tâches pour lesquelles personne n'a jamais écrit un tel algorithme, alors même que des humains les résolvent couramment : reconnaître qu'une image montre un chat, juger de la grammaticalité d'une phrase. Ce ne sont pas des tâches mal posées : on sait reconnaître une bonne réponse d'une mauvaise. Ce qui manque, c'est une façon d'*écrire* la règle. Personne ne peut décrire, en une suite finie d'instructions portant sur des intensités de pixels, ce qui distingue un chat d'un chien.

Nota

Le critère qui sépare ces deux catégories n'est pas « facile » contre « difficile », mais l'existence ou non d'une règle explicite dont on puisse prouver la correction. Il vaut la peine de noter, en anticipant sur la suite, qu'il est possible d'entraîner un système par apprentissage à deviner la dérivée d'une formule symbolique (traitée comme une tâche de traduction : une formule en entrée, une formule en sortie), avec une précision élevée sur des formules jamais vues, alors même que l'algorithme exact existe déjà pour ce problème. Ce fait, sur lequel on reviendra, montre que l'apprentissage n'est pas réservé aux problèmes sans solution connue : il devient seulement *facultatif* dès qu'une règle explicite existe, et *nécessaire* quand aucune n'est connue.

Le changement de paradigme consiste alors à ne plus écrire la règle soi-même, mais à la faire produire par une machine à partir d'exemples : on donne des paires (entrée, sortie correcte), par exemple des images déjà étiquetées « chat » ou « chien », et on demande à un algorithme de produire, à partir de ces exemples, une règle qui fonctionne aussi sur des entrées jamais vues. Chaque objet introduit dans ce qui suit répond à un besoin précis de ce programme.

2.2 Le cadre statistique de l'apprentissage

2.2.1 Représenter les entrées et les sorties

On a vu en 2.1 qu'il s'agit, étant donnée une entrée, de produire la sortie correcte associée, sans disposer d'une règle explicite pour le faire. On formalise à présent les ensembles dans lesquels vivent ces objets.

Définition (Problème de prédiction)

Un *problème de prédiction* est la donnée d'un couple d'ensembles non vides $(\mathcal{X}, \mathcal{Y})$. On appelle $\mathcal{X}$ l'**espace des entrées** du problème et $\mathcal{Y}$ son **espace des sorties** : les entrées possibles pour ce problème sont, par définition, les éléments de $\mathcal{X}$, et les sorties possibles les éléments de $\mathcal{Y}$. Ce qui distingue, pour une entrée $x \in \mathcal{X}$ donnée, une sortie $y \in \mathcal{Y}$ correcte d'une sortie incorrecte n'est pas encore précisé à ce stade ; ce sera l'objet de 2.2.2.

Un problème de prédiction $(\mathcal{X}, \mathcal{Y})$ est dit de **classification** si $\mathcal{Y}$ est fini (ses éléments étant alors identifiés, sans perte de généralité, à $\{1, \dots, K\}$ où $K = |\mathcal{Y}|$), et de **régression** si $\mathcal{Y} = \mathbb{R}$ (ou $\mathbb{R}^m$).

Exemple. « Chat ou chien » est un problème de classification avec $K = 2$, où $\mathcal{X} = [0, 1]^{H \times W \times 3}$ pour une résolution d'image $H \times W$ à trois canaux de couleur fixée une fois pour toutes. Prédire une température est un problème de régression, $\mathcal{Y} = \mathbb{R}$.

2.2.2 Pourquoi une loi de probabilité plutôt qu'une fonction

Nota

On dira qu'une relation entre $\mathcal{X}$ et $\mathcal{Y}$ est *fonctionnelle* si à chaque x correspond un unique y , c'est-à-dire si c'est le graphe d'une fonction $f : \mathcal{X} \rightarrow \mathcal{Y}$.

Deux faits empêchent la relation « entrée, bonne étiquette » d'être fonctionnelle en général : l'ambiguïté intrinsèque (deux occurrences identiques de x annotées différemment selon le contexte) et le bruit d'observation (erreur d'annotation). On remplace donc l'hypothèse d'une fonction f par celle, plus faible, d'une loi jointe $\mathcal{D}$ sur $\mathcal{X} \times \mathcal{Y}$, inconnue, fixée, jamais observée directement, seulement échantillonnée : $(X, Y) \sim \mathcal{D}$. Si $\mathcal{D}$ était connue, il n'y aurait rien à apprendre.

2.2.3 Espace des hypothèses, perte, risque

On cherche une fonction $h : \mathcal{X} \rightarrow \mathcal{Y}$ qui approche au mieux le comportement de $\mathcal{D}$. Deux objets sont nécessaires pour rendre cette recherche précise :

- un **espace des hypothèses** $\mathcal{H}$, l'ensemble des fonctions h parmi lesquelles on cherche (pour les réseaux de neurones, $\mathcal{H}$ sera l'ensemble des fonctions représentables par une architecture fixée, paramétrée par ses poids) ;
- une **fonction de perte** $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$, qui mesure l'écart entre une prédiction et la vraie valeur (perte 0-1, $\ell(\hat{y}, y) = \mathbb{1}[\hat{y} \neq y]$, pour la classification ; perte quadratique, $\ell(\hat{y}, y) = (\hat{y} - y)^2$, pour la régression).

Définition (Risque)

Le risque de $h \in \mathcal{H}$ est

$$R(h) = \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(h(X), Y)].$$

C'est la quantité qu'on veut minimiser, mais elle dépend de $\mathcal{D}$, donc elle n'est pas calculable.

Ce qu'on observe est un échantillon $S = ((x_1, y_1), \dots, (x_n, y_n))$ tiré i.i.d. selon $\mathcal{D}$.

Définition (Risque empirique)

$$\hat{R}_S(h) = \frac{1}{n} \sum_{i=1}^n \ell(h(x_i), y_i).$$

Convention (Principe ERM)

On choisit $\hat{h} \in \mathcal{H}$ minimisant $\hat{R}_S$ sur $\mathcal{H}$, au sens précisé par la définition 2.3.1 ci-dessous.

La question centrale, qu'il faut poser avant de continuer : pourquoi minimiser $\hat{R}_S(h)$ devrait-il dire quelque chose sur $R(h)$? Rien ne l'exclut *a priori* : on peut avoir $\hat{R}_S(h) = 0$ avec $R(h)$ arbitrairement mauvais (un h qui apprend l'échantillon par cœur). L'écart $R(h) - \hat{R}_S(h)$ et son contrôle (loi des grands nombres pour h fixé, VC-dimension ou complexité de Rademacher pour h choisi après coup) sont l'objet de la théorie statistique de l'apprentissage. Nous laissons ce contrôle de côté pour l'instant, et renvoyons le lecteur à [2] pour un traitement complet ; il n'est pas nécessaire à la compréhension de ce qui suit.

2.3 Optimisation

2.3.1 Le problème que l'optimisation doit résoudre

Le principe ERM ramène l'apprentissage à un problème de minimisation : trouver $\hat{h} \in \mathcal{H}$ qui minimise $\hat{R}_S(h)$. Pour que ce problème soit traitable algorithmiquement, il faut que $\mathcal{H}$ soit paramétré de façon exploitable. On suppose désormais que $\mathcal{H} = \{h_\theta : \theta \in \Theta\}$, où $\Theta \subseteq \mathbb{R}^p$ est un sous-espace de paramètres de dimension finie p , et $\theta \mapsto h_\theta$ une famille de fonctions fixée (l'architecture). Minimiser $\hat{R}_S(h)$ sur $\mathcal{H}$ revient alors à minimiser

$$\mathcal{L}(\theta) := \hat{R}_S(h_\theta) = \frac{1}{n} \sum_{i=1}^n \ell(h_\theta(x_i), y_i)$$

sur Θ , un problème d'optimisation en dimension finie, mais avec p potentiellement immense (des millions, voire des milliards de paramètres pour un Transformer), ce qui exclut d'emblée les méthodes qui nécessitent d'inverser une matrice de taille $p \times p$ (comme la méthode de Newton) ou d'énumérer Θ .

Définition (Minimiser)

Soit $f : \Theta \rightarrow \mathbb{R}$ et $\Theta \neq \emptyset$. On dit que $\theta^* \in \Theta$ *minimise* f sur Θ si $f(\theta^*) \leq f(\theta)$ pour tout $\theta \in \Theta$. On note alors $\theta^* \in \arg \min_{\theta \in \Theta} f(\theta)$.

Remarque

Rien ne garantit qu'un tel θ^* existe. Ce que garantit toujours l'analyse, c'est l'existence de la borne inférieure $\inf_{\theta \in \Theta} f(\theta) \in \mathbb{R} \cup \{-\infty\}$ (un ensemble non vide de réels minorés admet toujours une borne inférieure, par la propriété de la borne inférieure de $\mathbb{R}$), mais cette borne peut ne jamais être atteinte par aucun point de Θ (exemple simple : $f(\theta) = e^{-\theta}$

sur $\Theta = \mathbb{R}$, $\inf f = 0$, jamais atteint). L'existence effective d'un minimiseur est garantie, par exemple, si Θ est compact et f continue, par le **théorème des bornes atteintes** (théorème de Weierstrass) : f continue sur un compact K atteint son minimum et son maximum sur K . Ce n'est en général **pas** le cas ici : $\Theta = \mathbb{R}^p$ n'est pas compact.

Convention adoptée pour la suite : quand on écrit « $\hat{h}$ minimise $\hat{R}_S(h)$ sur $\mathcal{H}$ », il faut comprendre cette expression au sens faible suivant : on ne cherche pas un point qui atteint exactement l'infimum, mais un point θ tel que $\mathcal{L}(\theta)$ soit arbitrairement proche de $\inf_{\Theta} \mathcal{L}$, ce que produit précisément un algorithme itératif comme la descente de gradient (sans garantie que cette limite soit l'infimum global si $\mathcal{L}$ n'est pas convexe).

2.3.2 La descente de gradient

Idée. Si $\mathcal{L}$ est différentiable, $-\nabla \mathcal{L}(\theta)$ indique, localement, la direction de plus forte décroissance de $\mathcal{L}$ en θ . On construit donc une suite $(\theta_t)_{t \geq 0}$ par

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t),$$

où $\eta > 0$ est le pas d'apprentissage (*learning rate*).

Proposition (Décroissance locale)

Pour η suffisamment petit, $\mathcal{L}(\theta_{t+1}) \leq \mathcal{L}(\theta_t)$, avec égalité seulement si $\nabla \mathcal{L}(\theta_t) = 0$.

Démonstration. Développement de Taylor à l'ordre 1 :

$$\mathcal{L}(\theta_t - \eta \nabla \mathcal{L}(\theta_t)) = \mathcal{L}(\theta_t) - \eta \|\nabla \mathcal{L}(\theta_t)\|^2 + o(\eta),$$

donc pour η petit le terme dominant est $-\eta \|\nabla \mathcal{L}(\theta_t)\|^2 \leq 0$. □

Remarque

Ceci garantit une décroissance *locale*, pas la convergence vers un minimum global. $\mathcal{L}$, construite à partir d'un réseau de neurones, est en général non convexe : la descente de gradient peut converger vers un minimum local, un point-selle, ou un plateau. C'est un fait empirique documenté que ce n'est en général pas rédhibitoire en pratique pour les grands réseaux [1], mais aucune garantie théorique de proximité au minimum global n'existe dans ce cadre général.

2.3.3 Pourquoi la descente stochastique (SGD)

Calculer $\nabla \mathcal{L}(\theta_t)$ exactement demande de parcourir les n termes de la somme $\hat{R}_S$, ce qui devient coûteux quand n est grand. On observe que $\nabla \mathcal{L}(\theta) = \frac{1}{n} \sum_i \nabla_{\theta} \ell(h_{\theta}(x_i), y_i)$ est lui-même une moyenne : on peut donc l'estimer sans biais à partir d'un sous-ensemble $B \subset \{1, \dots, n\}$ (un *mini-batch*) tiré aléatoirement, en calculant $\frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell(h_{\theta}(x_i), y_i)$.

Proposition (Estimateur sans biais)

$$\mathbb{E}_B \left[\frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell(h_{\theta}(x_i), y_i) \right] = \nabla \mathcal{L}(\theta).$$

Démonstration. Par linéarité de l'espérance et le fait que chaque indice a la même probabilité d'être tiré. $\square$

2.3.4 Calculer $\nabla_{\theta} \mathcal{L}$: la rétropropagation

Reste le problème calculatoire : h_{θ} pour un réseau de neurones est une composition de nombreuses fonctions élémentaires (couches), chacune paramétrée par une partie de θ . Calculer $\nabla_{\theta} \mathcal{L}$ à la main, terme par terme, serait ingérable dès que le nombre de couches grandit.

Le fait mathématique unique qui résout tout est la règle de la chaîne. Si $h_{\theta} = f_L \circ f_{L-1} \circ \dots \circ f_1$, où chaque f_k dépend d'un sous-vecteur de paramètres θ_k , alors la dérivée de $\mathcal{L}$ par rapport à θ_k se calcule en propageant les dérivées de la sortie vers l'entrée, couche par couche, chaque couche ne calculant que la dérivée locale de sa propre fonction. C'est ce qu'on appelle la **rétropropagation** [3] : ce n'est pas un algorithme d'apprentissage à part entière, seulement une organisation efficace (en temps linéaire en le nombre de couches, par un unique parcours arrière du graphe de calcul) du calcul de la règle de la chaîne sur une composition profonde.

Références

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [2] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. MIT Press, 2nd edition, 2018.
- [3] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323 :533–536, 1986.